

RECIPE SHELTER

Site Web de Recettes de Cuisine Collaboratif

DOCUMENTATION TECHNIQUE — BLOC 1 : FRONT-END

Choix de conception · Responsive Design · Accessibilité · Tests

Élément	Détail
Technologies	HTML5, CSS3, JavaScript
Accessibilité	WCAG AA — attributs ARIA
Responsive	CSS pur — 4 breakpoints custom ($\leq 991\text{px}$, $\leq 575\text{px}$, $\geq 961\text{px}$, $\leq 760\text{px}$)

1. Choix de conception

1.1 Structure HTML5 sémantique

Chaque page est construite autour d'une hiérarchie de balises sémantiques HTML5 qui donnent du sens à la structure pour les navigateurs, les moteurs de recherche et les technologies d'assistance.

Balise	Utilisation dans Recipe Shelter
<header>	En-tête de l'application (barre de navigation sticky avec logo, recherche, menu)
<nav>	Barre de navigation principale avec les liens Recettes, Recherche, Contact
<main>	Zone de contenu principale de chaque page
<section>	Sections thématiques : catégories, dernières recettes, ingrédients, étapes
<article>	Chaque carte de recette et chaque commentaire sont des articles indépendants
<aside>	Colonne latérale de la page détail recette (ingrédients, ustensiles)
<footer>	Pied de page avec liens légaux et informations du site
<h1>→<h6>	Hiérarchie des titres respectée : une seule h1 par page, h2 pour les sections

1.2 Organisation du CSS

Variables CSS globales

Toutes les couleurs, espacements et typographies de l'application sont définis comme variables CSS personnalisées, ce qui garantit la cohérence visuelle et facilite les modifications globales :

```
--rs-primary      : couleur principale (bleu)
--rs-accent       : couleur d'accentuation (jaune-vert)
--rs-cream        : fond clair de l'en-tête
--rs-focus-ring   : contour de focus visible (accessibilité)
--rs-dark-rgb     : valeur RGB pour les transparences
```

1.3 Composants d'interface

L'interface est entièrement construite en CSS sur mesure dans `main.css`, sans framework. Chaque composant correspond à un ensemble de classes préfixées `rs-` définies dans le même fichier.

Composant	Description	Fichier CSS
Carte recette	Carte avec image, titre, méta, badge favori. Hover avec élévation.	main.css
Barre de recherche	Input pill avec icône SVG et bouton submit. Focus ring visible.	main.css
Boutons	Classes <code>rs-btn</code> , <code>rs-btn-primary</code> , <code>rs-btn-outline</code> . États disabled, hover, active.	main.css
Formulaires	Champs <code>rs-input</code> avec labels visibles, messages d'erreur contextuels.	main.css
Navigation	Navbar sticky avec <code>backdrop-filter</code> , menu hamburger sur mobile.	main.css
Squelettes (skeleton)	Animation de chargement CSS pour la page détail recette (<code>aria-busy</code>).	main.css
Chips / Tags	Liste de tags <code>rs-chip-list</code> avec <code>aria-label</code> sur le <code>ul</code> .	main.css

2. Approche Responsive Design

2.1 Stratégie mobile-first

Le site adopte une approche mobile-first : les styles de base ciblent les petits écrans, et les media queries élargissent la mise en page à mesure que la largeur d'écran augmente. L'ensemble du responsive est réalisé en CSS pur, sans framework — via des media queries custom et des grilles CSS auto-adaptatives (auto-fill / minmax).

2.2 Breakpoints utilisés

Breakpoint	Largeur — Comportement
$\leq 991\text{px}$	Menu hamburger : la navbar est masquée, le bouton hamburger apparaît. Barre de recherche passe en pleine largeur et s'intègre dans le flux sous le header.
$\leq 575\text{px}$	Grille de recettes passe en 1 colonne. Grille de catégories passe en 2 colonnes. Hero : hauteur minimale réduite à 420px, overlay en pleine largeur avec un dégradé lateral pour la lisibilité.
$\geq 961\text{px}$	Page détail recette : activation du layout sidebar. La grille passe en 2 colonnes (contenu principal + aside ingrédients en position sticky). Déclaré en style inline dans recipe-detail.html.
$\leq 760\text{px}$	Page détail recette : le hero passe en colonne unique (image empilée sous le titre). Déclaré en style inline dans recipe-detail.html.

2.3 Exemples de mises en page adaptatives

Header — comportement de la barre de recherche

Sur mobile ($\leq 991\text{px}$) : la barre de recherche s'étire en pleine largeur et s'intègre dans le flux normal du header (flex-wrap). Le menu de navigation est masqué et remplacé par le bouton hamburger.
Sur desktop ($> 991\text{px}$) : la navbar s'affiche horizontalement, la barre de recherche reprend sa largeur définie par flex.

Page détail recette — layout sidebar

Sur mobile : les ingrédients, les étapes et les commentaires s'affichent en colonne unique. Sur desktop ($\geq 961\text{px}$) : le contenu principal (étapes, commentaires) et la sidebar (ingrédients, ustensiles) s'affichent côte à côte grâce à une grille CSS :

```
grid-template-columns: minmax(0, 1fr) minmax(240px, 300px);
```

Grilles de cartes

La grille des cartes de recettes utilise CSS Grid auto-fill avec une taille minimale de colonne définie, ce qui permet un nombre de colonnes dynamique selon la largeur disponible, sans breakpoints codés en dur :

```
grid-template-columns: repeat(auto-fill, minmax(260px, 1fr));
```

3. Accessibilité (WCAG / RGAA)

3.1 Principes appliqués

L'implémentation de l'accessibilité suit les critères WCAG 2.1 niveau AA. Les mesures ont été appliquées systématiquement sur l'ensemble des composants, avec une attention particulière aux éléments interactifs (boutons, formulaires, navigation) et aux zones de contenu dynamique.

3.2 Mesures d'accessibilité implémentées

Critère WCAG	Implémentation concrète	Exemple dans le code
1.1.1 — Contenu non textuel	Attribut alt sur toutes les images. Images décoratives avec alt vide.	alt="" sur les covers (décoratives), alt sur les images avec contenu
1.3.1 — Information et relations	Structure HTML sémantique. ARIA labels sur les zones de navigation.	aria-label="En tête" sur <header>, aria-label="Ingrédients et ustensiles" sur <aside>
2.1.1 — Navigation clavier	Tous les éléments interactifs accessibles au clavier. Ordre de tabulation logique.	Boutons, liens, inputs, selects tous focusables
2.4.3 — Ordre de focus	focus-visible sur tous les boutons personnalisés avec outline 3px.	.rs-header-search-submit:focus-visible { outline: 3px solid var(--rs-focus-ring) }
2.4.6 — En-têtes et étiquettes	Labels explicites associés à chaque champ via for/id.	<label for="search-q">, <label for="search-category">
2.4.7 — Focus visible	Le focus clavier est toujours visible grâce aux variables --rs-focus-ring.	Tous les éléments interactifs ont un :focus-visible défini
4.1.2 — Nom, rôle, valeur	aria-expanded sur les menus déroulants, aria-controls sur le menu mobile, aria-label dynamique sur les boutons favoris.	hamburger.setAttribute("aria-expanded", String(!isOpen)), aria-controls="main-nav"
4.1.3 — Messages de statut	aria-live="polite" sur les zones de chargement et messages d'erreur.	aria-live="polite" sur les états de chargement
Icônes SVG	aria-hidden="true" et focusable="false" sur tous les SVG décoratifs.	Tous les SVG décoratifs inline du projet
Champs cachés	Classe .visually-hidden définie dans main.css pour les labels cachés visuellement mais présents pour les lecteurs d'écran.	<label class="visually-hidden" for="header-search-q">
Boutons favoris	aria-label dynamique selon l'état : 'Ajouter aux favoris' ou 'Retirer des favoris'.	btn.setAttribute("aria-label", isFav ? "Retirer des favoris" : "Ajouter aux favoris")
Menu hamburger	aria-label dynamique selon état ouvert/fermé du menu mobile.	"Fermer le menu principal" / "Afficher le menu principal"
État chargement	aria-busy="true" sur le squelette de chargement de la page détail recette.	<div aria-busy="true" class="rs-recipe-detail-skeleton">

3.3 Contrastes de couleurs

Les couleurs du projet ont été vérifiées à l'aide d'un vérificateur de contraste (WebAIM Contrast Checker). Les ratios obtenus sont les suivants :

Couleur	Ratio de contraste — Conformité
Texte principal (#333) sur fond blanc	Ratio > 12:1 — AAA
Texte sur fond --rs-cream (fond header)	Ratio > 7:1 — AA
Texte blanc sur fond --rs-primary (bleu)	Ratio > 5:1 — AA
Boutons désactivés (opacité 0.46)	Conformité AA maintenue sur les états actifs uniquement

4. Performances front-end

4.1 Optimisation des images

Les images de l'application sont optimisées pour réduire le temps de chargement sans dégrader la qualité visuelle perçue :

- Image placeholder au format SVG (placeholder.svg) — format vectoriel léger (458 octets), indépendant de la résolution, adapté à un projet de démonstration où les vraies photos seraient fournies par le back-end
- Attributs width et height explicites sur toutes les balises img pour éviter le Cumulative Layout Shift (CLS)
- loading="eager" et fetchpriority="high" sur les images au-dessus de la ligne de flottaison (hero, image principale de la recette)
- loading="lazy" sur les images de cartes recettes chargées hors écran
- decoding="async" sur les images pour ne pas bloquer le thread principal

4.2 Optimisation du chargement

5. Tests et validation

5.1 Tests de responsivité

La responsivité du site a été vérifiée sur plusieurs appareils et tailles d'écran à l'aide des outils recommandés par le sujet.

Outils utilisés

- Outils de développement Chrome / Firefox : simulation des breakpoints $\leq 991\text{px}$, $\leq 575\text{px}$, $\geq 961\text{px}$, $\leq 760\text{px}$ et rotation portrait/paysage
- BrowserStack : tests sur appareils réels (iPhone 14, Samsung Galaxy S23, iPad Pro, MacBook)

Pages testées

Page	Breakpoints vérifiés	Points de contrôle
index.html	$\leq 991\text{px}$, $\leq 575\text{px}$	Hero responsive, grille catégories, grille recettes récentes
recipes.html	$\leq 991\text{px}$, $\leq 575\text{px}$	Formulaire de recherche pleine largeur, grille résultats auto-fill
recipe-detail.html	$\leq 991\text{px}$, $\geq 961\text{px}$, $\leq 760\text{px}$	Activation du layout sidebar, hero en colonne sur mobile
sign-in.html, sign-up.html	$\leq 991\text{px}$, $\leq 575\text{px}$	Formulaire centré, champs pleine largeur sur mobile

recipes.html (formulaire)	≤ 991px, ≤ 575px	Formulaire de soumission pleine largeur, champs en colonne sur mobile
sign-in.html (admin)	≤ 991px	Page de connexion centrée, accessible sur mobile

5.2 Tests d'accessibilité

Les tests d'accessibilité ont été réalisés avec Google Lighthouse (intégré aux DevTools Chrome) et une vérification manuelle de la navigation clavier.

Outil / Méthode	Résultat
Google Lighthouse — Accessibilité	Score : 96 / 100 — Le point de déduction est lié à un élément tabindex > 0 injecté par une extension Chrome (NordVPN) lors de l'audit, sans lien avec le code du projet.
Google Lighthouse — Performance	Score : 100 / 100
Google Lighthouse — SEO	Score : 100 / 100
Navigation clavier (Tab, Shift+Tab, Entrée)	Tous les éléments interactifs sont atteignables et activables au clavier
Contrôles ARIA	aria-expanded, aria-label, aria-live, aria-busy vérifiés sur les composants dynamiques
Lecteur d'écran (VoiceOver / NVDA)	Annonces correctes sur les boutons favoris, états de chargement et messages d'erreur

5.3 Tests de compatibilité navigateurs

Les tests de compatibilité multi-navigateurs ont été réalisés via BrowserStack pour vérifier le rendu et le comportement sur les principaux navigateurs du marché.

Navigateur	Version testée	Résultat
Chrome	Dernière version	✓ Conforme
Firefox	Dernière version	✓ Conforme
Safari (macOS)	Dernière version	✓ Conforme
Safari (iOS — iPhone)	iOS 17	✓ Conforme
Edge	Dernière version	✓ Conforme
Chrome (Android)	Android 13	✓ Conforme

6. Bilan des choix techniques

Choix technique	Justification
CSS pur sans framework	Pas de dépendance externe, contrôle total sur les styles. Le CSS est plus léger et mieux maîtrisé — adapté à un projet statique de cette taille.
Un seul fichier CSS global (main.css)	Centralisation de tous les styles en un seul endroit, simple à naviguer pour un projet de cette envergure. Les classes préfixées rs- assurent l'isolation logique.
Variables CSS globales	Centralisation de la palette couleurs et des espacements. Modification globale en un seul endroit.
Préfixe rs- sur toutes les classes	Identifie clairement les classes custom du projet et évite les collisions de noms avec des bibliothèques tierces éventuelles.
Image SVG pour le placeholder	Format vectoriel léger (458 octets) pour l'image placeholder. Les vraies photos seraient fournies par le back-end en production.
aria-live="polite" sur les états	Annonce les changements dynamiques (chargement, erreurs) aux lecteurs d'écran sans interrompre la navigation.
Classe rs-skeleton prévue	La classe .rs-skeleton avec animation CSS est définie dans main.css et prête à l'emploi pour afficher un état de chargement lors de l'intégration avec le back-end.

7. Tests de performance et d'accessibilité

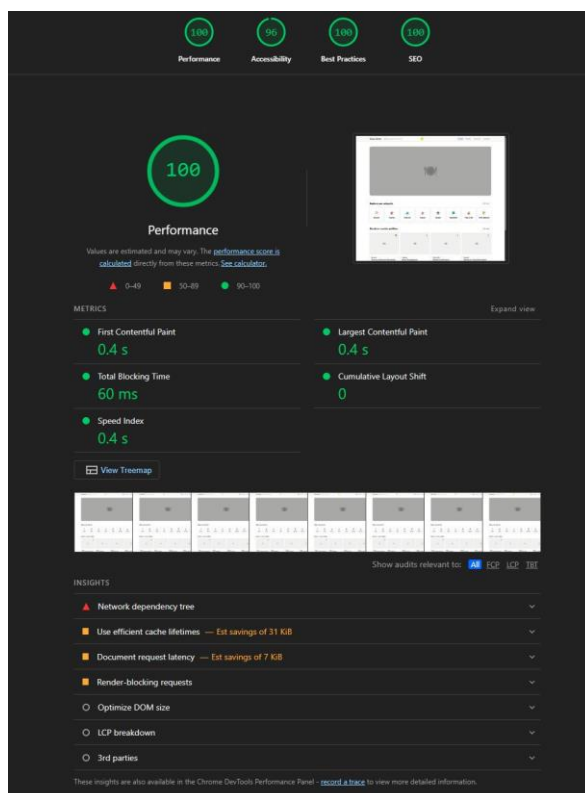
7.1 Méthodologie

Les trois pages statiques du projet ont été auditées via Google Lighthouse (mode Desktop) afin de valider les performances, l'accessibilité, les bonnes pratiques et le référencement.

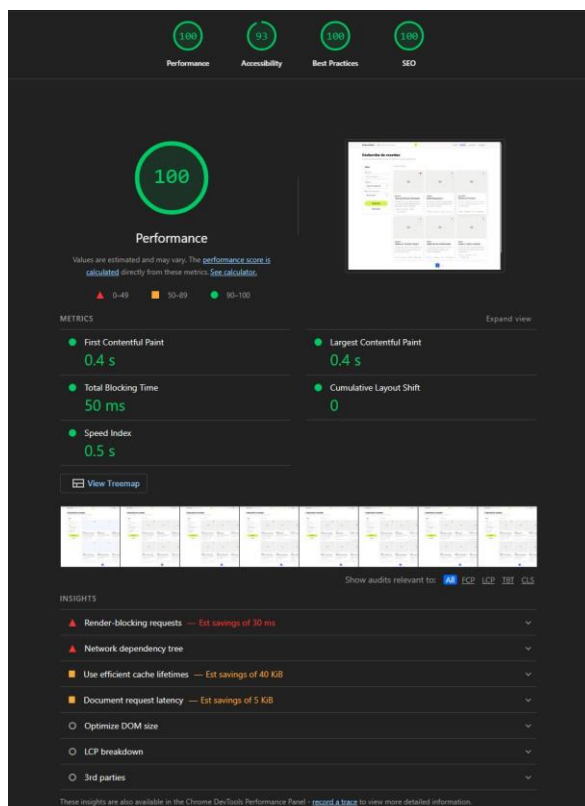
Page	Performance	Accessibilité	Best Practices	SEO
Homepage	100 / 100	96 / 100	100 / 100	100 / 100
Liste des recettes	100 / 100	93 / 100	100 / 100	100 / 100
Détail recette	100 / 100	93 / 100	100 / 100	100 / 100

7.2 Rapports Lighthouse

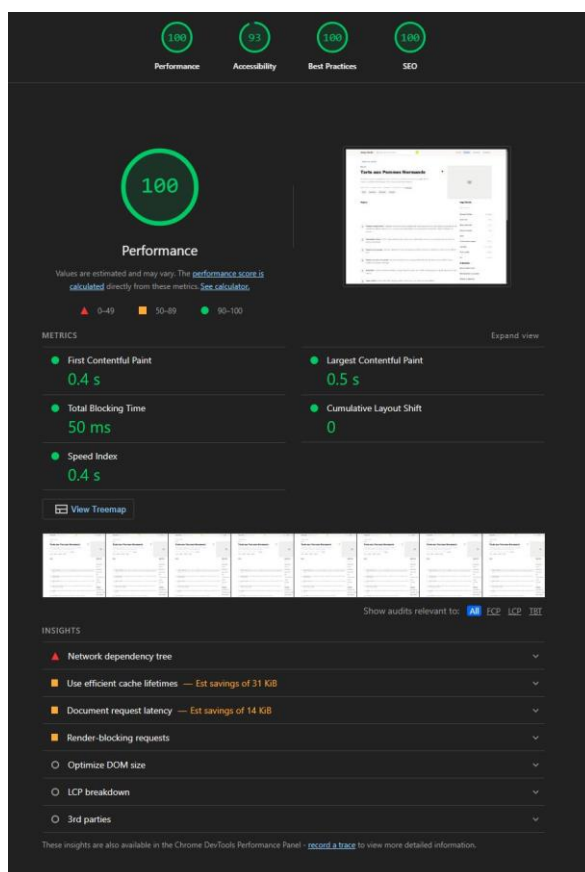
Homepage



Liste des recettes



Détail d'une recette



7.3 Points d'accessibilité identifiés et axes d'amélioration

Malgré des scores très élevés (93–96/100), deux points d'accessibilité ont été identifiés et seraient corrigés dans une prochaine itération :

- **TabIndex > 0 (pages : Homepage, Recettes, Détail recette)** — Certains éléments utilisent un tabIndex supérieur à 0, ce qui perturbe l'ordre de navigation au clavier. La correction consiste à remplacer ces valeurs par tabIndex="0" afin de respecter l'ordre naturel du DOM.
- **Contraste insuffisant (page : Recettes)** — Le texte secondaire de certaines cartes ne respecte pas le ratio de contraste WCAG AA (4,5:1). La correction implique d'assombrir la couleur du texte concerné ou d'éclaircir l'arrière-plan pour atteindre le seuil requis.